

states, you just set the skin's objects, properties, and styles without any state notation. But you must declare the states or a runtime error is generated when the skin is bound to the application at runtime.

First look at the host component's documentation to find out which states are required. Then open the Flex help system and search for spark.components.Application. After opening the help screen for the Application component, click Skin States. If necessary, click Show Inherited Skin States to see the states inherited from the Application component's superclasses. The Application component requires the skin states disabled and normal. Now return to the new skin component file in Flash Builder and add the following code after the default <fx:Declarations> element:

- `<s:states>`
- `<s:State name="normal" />`
- `<s:State name="disabled" />`
- `</s:states>`

In the skin code, you'll be able to designate inclusion or exclusion of particular objects depending on which state the application is in with the `includeIn` and `excludeFrom` properties. For example, the following Group is only included in the normal state:

- `<s:Group id="normalGroup" minWidth="0" minHeight="0"`
- `includeIn="normal" >`
- `... visual controls and graphics ...`
- `</s:Group>`

You're also able to control individual properties and styles with dot notation. The following style overrides the skin's default `chromeColor` setting only when the application's state is disabled:

- `chromeColor.disabled="#EEEEEE"`

Spark components typically implement one or more required skin parts. These are typically used for specific functionality in the component. For example, the Application component requires a skin part named `contentGroup` that must implement the Group component or one of its subclasses. This is the actual container used to lay out the visual children declared with MXML in the Application. Or in button skins, the `labelDisplay` skin part is a TextBase object that will display the label of any button that uses that skin. If you don't declare the required skin parts in your custom skin, the application fails at runtime when it tries to use the skin part; in the case of `contentGroup`, it fails when it tries to add visual controls declared with MXML to the group upon application startup.